For loop and Arrays

for Loop

- for (*initialization*; *condition*; *update*)
  { *statement;* }

- Example:

```
for (int i = 1; i <= n; i++)
{
double interest = balance * rate / 100;
balance = balance + interest;
}
```

while loop as for loop

- Equivalent to

```
initialization;
while (condition)
 {
   statement;
   update;
 }
```

```
 for (int i = 1; i <= n; i++)
{ statement;
}
```

Self Check

- How many times does the following for loop execute?

```
for (i = 0; i <= 10; i++)
   System.out.println(i);
```

- Rewrite the above for loop as a while loop

Answers

- 11 times
-
```
int i = 0;
while (i <= 10)
{
 System.out.println(i);          //  {  System.out.println(i++); }
i++;
}
```

Common Errors

- Example:

```
sum = 0;
for (i = 1; i <= 10; i++);
    sum = sum + i;
    System.out.println(sum);
```

Delete semi colon

Use parenthesis if more than one statement

Nested Loops

- Some triangle patterns

```
*
**
***
****
```

(1)

```
   *
  ***
 *****
*******
```

(2)

```
    *
   ***
  *****
 *******
*****
***
*
```

(3)

Self Check

- What is the value of n after the following nested loops?

```
int n = 0;
for (int i = 1; i <= 5; i++)
   for (int j = 0; j < i; j++)
      n = n + j;
```

Answers

- 20

Array

- Sequence of values of the same type
- Construct array:

  new double[10]
- Store in variable of type double[]

  double[] data = new double[10];
- When array is created, all values are initialized depending on array type:
  - Numbers: 0
  - Boolean: false
  - Object References: null

Element access of an array

- Use arrayname[i] to access an element
  data[2] = 29.95;
  data[5] = 19.05;
  data[8] = 9.91;

Self Check

- What elements does the data array contain after the following statements?

```
int[ ] data = new int[10];
for (int i = 0; i < data.length; i++)
    data[i] = i * i;
```

Answers

- 0, 1, 4, 9, 16, 25, 36, 49, 64, 81, but not 100

Self Check

- What do the following program segments print? Or, if there is an error, describe the error and specify whether it is detected at compile-time or at run-time.
  - double[ ] a = new double[10];
    System.out.println(a[0]);
  - double[ ] b = new double[10];
    System.out.println(b[10]);
  - double[ ] c;
    System.out.println(c[0]);

Answers

- -
  - 0.0
  - a run-time error: array index out of bounds
  - a compile-time error: c is not initialized

Generic classes

```
class Solution<T>
{  T data;
   public static T getData()
    {
       return data;
 } }

Solution<String> S=new Solution<String>();
```

Array Lists

- The ArrayList class
  - manages a sequence of objects
  - Can grow and shrink as needed
  - supplies methods for many common tasks, such as inserting and removing elements
  - a generic class: **ArrayList<T>** collects objects of type T:

```
ArrayList<BankAccount> accounts = new
                                    ArrayList<BankAccount>();
accounts.add(new BankAccount(1001));
accounts.add(new BankAccount(1015));
accounts.add(new BankAccount(1022));
```

  - size() method yields number of elements

Retrieving Array List Elements

- Use get() method
- Index starts at 0
- BankAccount anAccount = accounts.get(2);

  // gets the third element of the array list
- Bounds error if index is out of range
- Most common bounds error:

  int i = accounts.size();

  anAccount = accounts.get(i); // Error

  // legal index values are 0. . .i-1

Adding and Removing elements

- set() method overwrites an existing value
  BankAccount anAccount = new BankAccount(1729);
  accounts.set(2, anAccount);

- add() adds a new value accounts.add(i, a)

```
X.add("red");     // ["red"]
X.add("blue");    // ["red" , "blue"]
X.add(1, "white"); // ["red" , "white", "blue"]
```

- remove method removes an element at an index
  accounts.remove(i)

Self Check

- How do you construct an array of 10 strings?
- An array list of strings?
- What is the content of names after the following statements?

```
ArrayList<String> names = new ArrayList<String>();
names.add("A");
names.add(0, "B");
names.add("C");
names.remove(1);
```

Answers

- new String[10];
- new ArrayList<String>();
- names contains the strings "B" and "C" at positions 0 and 1

Designing Classes

Choosing Classes

- A class represents a single concept from the problem domain
- Name for a class should be a noun that describes concept

- Concepts from mathematics: Point, Rectangle, Ellipse
- Concepts from real life: BankAccount, Student, Employee

Static Methods

- Every method must be in a class
- A static method is not invoked on an object

Quest: Suppose Java had no static methods. Then all methods of the Math class would be instance methods. How would you compute the square root of x?

Ans:
```
Math m = new Math();
y = m.sqrt(x);
```

Static Fields/variables

- A static field belongs to the class, not to any object of the class. Also called *class field*.

- public class BankAccount
  {
  . . .
  private double balance;
  private int accountNumber;
  private static int var = 1000; // Executed once, when class is loaded
  }

- call as BankAccount.var

- If var was not static, each instance of BankAccount would have its own value of var

Static variable

- All objects of its class, shares same static variable space
- If value changes, it will changed for every later call to the static variable (until further change)
- It can be think as a common shared variable
  - ```
    public BankAccount()     // constructor
    {
                                  // Generates next account number to be assigned
      var++;                      // Updates the static field
                                  // Assigns field to account number of this bank account
      accountNumber = var;        // Sets the instance field
    }
    ```
  - Minimize the use of static fields. (Static final fields are ok.)
    ```

Scope of Local Variables

- Scope of variable: Region of program in which the variable can be accessed
- Scope of a local variable extends from its declaration to end of the block that encloses it
- Sometimes the same variable name is used in two methods:

```
public class RectangleTester
    {
    public static double area(Rectangle rect)
            {
            double r = rect.getWidth() * rect.getHeight();
            return r;
            }
    public static void main(String[] args)
            {
            Rectangle r = new Rectangle(5, 10, 20, 30);
            double a = area(r);
            System.out.println(r);
            }
    }
```

- These variables are independent from each other; their scopes are disjoint

Scope of Local Variables

- Scope of a local variable cannot contain the definition of another variable with the same name

```
Rectangle r = new Rectangle(5, 10, 20, 30);
if (x >= 0)
{
        double r = Math.sqrt(x);
        // Error–can't declare another variable named r here
        . . .
}
```

- However, can have local variables with identical names if scopes do not overlap

```
if (x >= 0)
{       double r = Math.sqrt(x);
        . . .
}
else   // Scope of r ends here
{       Rectangle r = new Rectangle(5, 10, 20, 30);
        // OK–it is legal to declare another r here   . . .
}
```

Scope of Class Members

- Private members have class scope: You can access all private members in any method of the class
- Public members have outside scope

  Math.sqrt
  harrysChecking.getBalance

- Inside a method, no need to qualify member variables or methods that belong to the same class

- An unqualified instance variable or method name refers to the this parameter

```
public class BankAccount
{
        public void transfer(double amount, BankAccount other)
        {
                withdraw(amount);     // i.e., this.withdraw(amount);
                other.deposit(amount);
}       }
```

Overlapping Scope

- A local variable can *shadow* a variable with the same name

- Local scope wins over class scope

```
public class Coin
{       . . .
        public double getExchangeValue(double exchangeRate)
        {           double value; // Local variable
                    . . .
                    return value;
        }
        private String name;
        private double value; // variable with the same name
}
```

- Access shadowed fields by qualifying them with the this reference

```
value = this.value * exchangeRate;
```

An example

```
class temp1
{ public double get()
  {    double value=10.0; // Local variable
        value = value + this.value;
       return value;
  }
  private String name;
  public double value=20.0; // Field with the same name
}

public class temp2
{ public static void main(String args[])
  {
        temp1 a=new temp1();
       System.out.println(a.get());
       System.out.println(a.value);
 }}
```

Output: 30
20

Organizing Related Classes into Packages

- Package: Set of related classes.
- To put classes in a package, you must place a line

  package packageName;

- the first instruction in the source file containing the classes Package name

Organizing Related Classes into Packages

Package	Purpose	Sample Class
java.lang	Language support	Math, String, Character
java.util	Utilities	Random
java.io	Input and output	PrintStream
java.awt	Abstract Windowing Toolkit	Color
java.applet	Applets	Applet
java.net	Networking	Socket
java.sql	Database Access	ResultSet
javax.swing	Swing user interface	Jbutton, JFrame
org.omg.CORBA	Common Object Request Broker Architecture	IntHolder

Importing Packages

- Can always use class without importing
  java.util.Scanner in = new java.util.Scanner(System.in);
- Tedious to use fully qualified name
- Import lets you use shorter class name
  import java.util.Scanner;
  . . .
  Scanner in = new Scanner(System.in)
- Can import all classes in a package
  import java.util.*;

- Never need to import java.lang
- You don't need to import other classes in the same package

Package Names and Locating Classes

- Use packages to avoid name clashes

  java.util.**Timer** vs. javax.swing.**Timer**

- Package names should be unambiguous

Self Check

- Which of the following are packages?
  - java
  - java.lang
  - java.util
  - java.lang.Math
- Can you write a Java program without ever using import statements?

Answers

- No Yes Yes No
- Yes–if you use fully qualified names for all classes, such as java.util.Random and java.awt.Rectangle